Design and Analysis of Algorithm

Project Final Report

Project Title: **Friend Recommendation System.**

Group Member:

Muhammad Azam (IU05-0124-0190)

Kashif Raza (IU05-0124-0303)

Course Instructor:

Miss Tooba Shaikh

Introduction

This project is a graph-based friend recommendation system implemented in Java using Swing for the graphical user interface. The system allows users to input friendships and recommends potential friends based on mutual connections. It demonstrates the use of graph data structures and algorithmic concepts in a real-world scenario.

Objective

- To recommend friends using a graph-based approach.
- To use HashMap and HashSet for efficient data storage.
- To provide a user-friendly GUI.
- To apply Design and Analysis of Algorithms concepts.

Methodology

1. Input friendships in the format user1 user2.
2. Store friendships as an undirected graph using HashMap and HashSet.
3. Identify friends of friends for recommendations.
4. Count mutual friends for ranking.
5. Sort recommendations in descending order.
6. Display results in the GUI.

Code

```
package Azam;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;

public class Azam extends JFrame {

    JTextArea inputwalaarea;
    JTextArea resultwalaarea;
    JTextField userfield;

    HashMap<String, HashSet<String>> graphdata;

    public Azam() {
        graphdata = new HashMap<>();

        setTitle("Friend Recommendation System");
        setSize(650, 550);
        setLayout(new BorderLayout());

        JLabel title = new JLabel("Graph Based Friend Recommendation System",
JLabel.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 18));
        add(title, BorderLayout.NORTH);

        JPanel centerpanel = new JPanel();
        centerpanel.setLayout(new GridLayout(1, 2));

        inputwalaarea = new JTextArea();
        inputwalaarea.setBorder(BorderFactory.createTitledBorder("Enter
friendships (user1 user2):"));
        JScrollPane scroll1 = new JScrollPane(inputwalaarea);
        centerpanel.add(scroll1);
```

```
        resultwalaarea = new JTextArea();
        resultwalaarea.setEditable(false);

resultwalaarea.setBorder(BorderFactory.createTitledBorder("Recommendations:"))
;
        JScrollPane scroll2 = new JScrollPane(resultwalaarea);
        centerpanel.add(scroll2);

        add(centerpanel, BorderLayout.CENTER);

        JPanel bottompanel = new JPanel();
        JLabel label1 = new JLabel("Enter user name:");
        bottompanel.add(label1);

        userfield = new JTextField(15);
        bottompanel.add(userfield);

        JButton checkbutton = new JButton("Recommend");
        bottompanel.add(checkbutton);

        add(bottompanel, BorderLayout.SOUTH);

        checkbutton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                processshurukaro();
            }
        });

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    void graphbanaya() {
        graphdata.clear();

        String rawdata = inputwalaarea.getText();
        String[] lines = rawdata.split("\n");

        for (String line : lines) {
            line = line.trim();
            if (line.length() == 0) continue;

            String[] users = line.split(" ");
            if (users.length == 2) {
                String user1 = users[0];
                String user2 = users[1];

                graphdata.putIfAbsent(user1, new HashSet<>());
                graphdata.putIfAbsent(user2, new HashSet<>());

                graphdata.get(user1).add(user2);
                graphdata.get(user2).add(user1);
```

```
            }
        }
    }

    void processshurukaro() {
        graphbanaya();
        resultwalaarea.setText("");

        String currentuser = userfield.getText().trim();

        if (!graphdata.containsKey(currentuser)) {
            resultwalaarea.setText("User not found in system!");
            return;
        }

        HashMap<String, Integer> suggestionmap = new HashMap<>();
        HashSet<String> currentuserfriends = graphdata.get(currentuser);

        for (String friend : currentuserfriends) {
            HashSet<String> friendsoffriend = graphdata.get(friend);

            for (String potentialfriend : friendsoffriend) {
                if (potentialfriend.equals(currentuser)) continue;
                if (currentuserfriends.contains(potentialfriend)) continue;

                int count = mutualfriendcount(currentuser, potentialfriend);
                suggestionmap.put(potentialfriend, count);
            }
        }

        ArrayList<String> userlist = new ArrayList<>(suggestionmap.keySet());
        for (int i = 0; i < userlist.size(); i++) {
            for (int j = i + 1; j < userlist.size(); j++) {
                String name1 = userlist.get(i);
                String name2 = userlist.get(j);

                int val1 = suggestionmap.get(name1);
                int val2 = suggestionmap.get(name2);

                if (val2 > val1) {
                    userlist.set(i, name2);
                    userlist.set(j, name1);
                }
            }
        }

        StringBuilder finaloutput = new StringBuilder();
        for (String name : userlist) {
            int score = suggestionmap.get(name);
            finaloutput.append(name).append(" -> mutual friends:
").append(score).append("\n");
        }
```

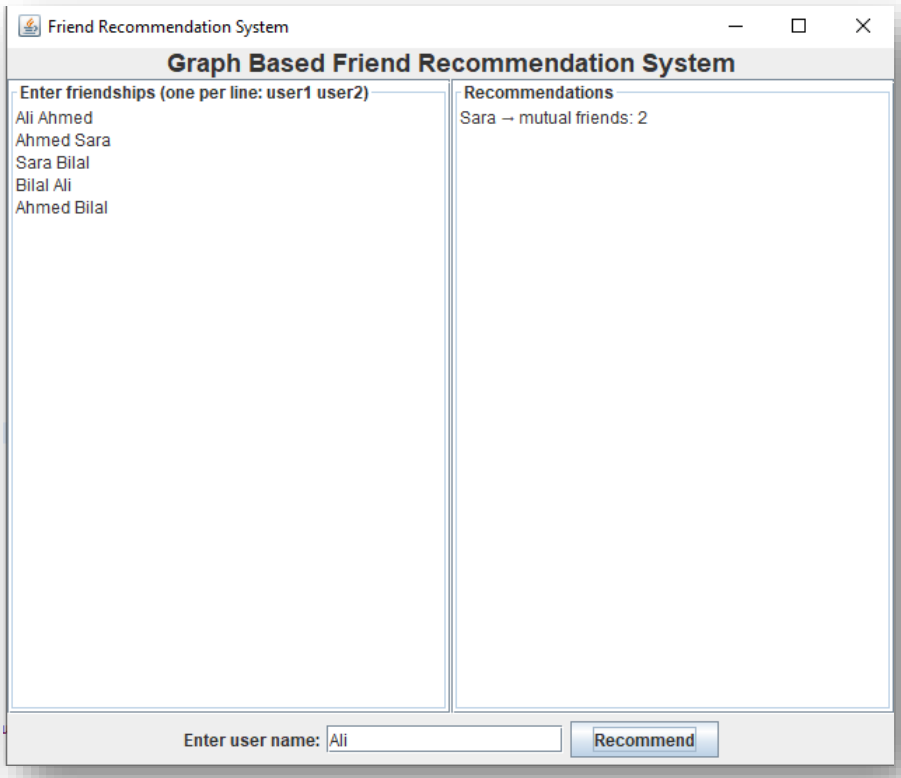
```java
        if (finaloutput.toString().isEmpty()) {
            resultwalaarea.setText("No recommendations found.");
        } else {
            resultwalaarea.setText(finaloutput.toString());
        }
    }

    int mutualfriendcount(String u1, String u2) {
        HashSet<String> set1 = graphdata.get(u1);
        HashSet<String> set2 = graphdata.get(u2);
        int count = 0;

        for (String s : set1) {
            if (set2.contains(s)) count++;
        }
        return count;
    }

    public static void main(String[] args) {
        new Azam();
    }
}
```

Output:



Design and Analysis of Algorithms Concepts Used

- Graph Representation (Undirected Graph)
- Hashing for fast lookup
- Brute Force / Iterative traversal
- Comparison-based Sorting
- Greedy / Heuristic approach

Conclusion

This project successfully demonstrates a friend recommendation system using graph data structures.
The system analyzes user relationships to suggest potential friends based on mutual connections.
It effectively applies concepts of Design and Analysis of Algorithms such as graph traversal and hashing.
Java Swing provides an interactive and user-friendly interface for input and output.
The use of efficient data structures improves performance and accuracy.
The project helps in understanding real-world applications of algorithms.
It also highlights the importance of logical input for correct output.
The system is scalable and can be expanded for large networks.
This project strengthens practical programming and problem-solving skills.
Overall, it is a successful implementation of algorithmic concepts in a social networking scenario.